

Unexpected Training Data Storage in `sklearn.feature_extraction.text.TfidfVectorizer` in [scikit-learn/scikit-learn](#)

✓ Valid

Reported on Apr 10th 2024

Description

TF-IDF is one of the simplest and well known technique to transform text data to vectors for ML models to consume. Scikit-learn is providing the implementation for this technique In `sklearn.feature_extraction.text.TfidfVectorizer`. A bug has been explored in this object that results in storing all the tokens in the training data, more than the required subset for this technique to operate. This flaw leads the path to leaking sensitive data. In theory and practice, the attribute causing the leakage does not need to store the mentioned data.

Proof of Concept

We follow a simple procedure with the default arguments except that we limit the vocabulary size. Limiting vocabulary is a very common setting hence provided by the library. The expected behaviour is that the object stores the frequent tokens, and discards the rest after the fitting process. In theory and practice, the vectorizer only needs the vocabulary and the rest of the possible tokens will be simply non needed, hence should be discarded.

We see a different behaviour than expected in this object. While the object correctly forms the required vocabulary, it stores the rest of the tokens in the ``stop_words_`` attribute. Therefore stores the entire unique tokens that have been passed in the fitting operation. Below we demonstrate this:

```
# └─$ pip freeze | grep pandas
# pandas==2.2.1
import pandas as pd
# └─$ pip freeze | grep scikit-learn
# scikit-learn==1.4.1.post1
from sklearn.feature_extraction.text import TfidfVectorizer

if __name__ == '__main__':
    # Fitting the vectorizer will save every token presented
    vectorizer = TfidfVectorizer(
        max_features=2,
        # min_df=2/6 # Same results occur with different ways of limiting the voca
    ).fit(
        pd.Series([
```

```

        "hello", "world", "hello", "world", "secretkey", "password123"
    ])
)
# Expected storage for frequent tokens
print(vectorizer.vocabulary_) # {'hello': 0, 'server': 1}
# Unexpected data leak
print(vectorizer.stop_words_) # {'password123', 'secretkey'}

```

We demonstrate below that the storage in the `stop_words_` attribute is unnecessary. Nullifying the attribute will give the same results:

```

# ↳$ pip freeze | grep pandas
# pandas==2.2.1
import pandas as pd
# ↳$ pip freeze | grep scikit-learn
# scikit-learn==1.4.1.post1
from sklearn.feature_extraction.text import TfidfVectorizer

if __name__ == '__main__':
    # Fitting the vectorizer will save every token presented
    vectorizer = TfidfVectorizer(
        max_features=2,
        # min_df=2/6 # Same results occur with different ways of limiting the voca
    ).fit(
        pd.Series([
            "hello", "world", "hello", "world", "secretkey", "password123"
        ])
    )
    # Expected storage for frequent tokens
    print(vectorizer.vocabulary_) # {'hello': 0, 'server': 1}
    # Unexpected data leak
    print(vectorizer.stop_words_) # {'password123', 'secretkey'}

    # Wiping-out the stop_words_ attribute does not change the behaviour
    print(vectorizer.transform(["hello world"]).toarray()) # [[0.70710678 0.707106
    vectorizer.stop_words_ = None
    assert vectorizer.stop_words_ is None
    print(vectorizer.transform(["hello world"]).toarray()) # [[0.70710678 0.707106

```

Impact

The vectorizers must be fit in the training time. The most likely and reasonable thing to do is to store it beside the ML models. While data exfiltration from ML model is challenging and a current research field, extracting data from the vectorizer is demonstrated to be trivial. An adversary who gets access to the vectorizer object will also have access to the tokens used in the training.

Depending on the use-case, the impact will vary from none to critical. For instance if the vectorizer is fit on the public documents, then the leakage will not reveal anything interesting, as the data is already present in public domain. In this instance the impact will be none to low. However if the vectorizer is fit on the documents that contains sensitive information like

confidential company info the impact may be high. If the vectorizer is fit on secrets like keys, passwords, and so on, the impact of the leakage may be critical.

Proposed CVSS

We propose the **CVSS: 5.3, Medium** due to following:

Attack Vector:(N)etwork The network access will allow accessing the vectorizer access.

Attack Complexity: (H)igh (1) The attacker needs to be familiar with the ML techniques. The attackers may not be aware of the potential ML vulnerabilities. (2) As (a) Unique tokens are saved with no order guarantee using python Set data structure, and (2) depending on the configuration the sensitive data may be fragmented into different tokens, the attacker may have to reconstruct the secrets or sensitive data.

Privileges Required: (L)ow Read access to the place where the vectorizer is stores is sufficient.

User Interaction: (N)one

Scope: (U)nchanged

Confidentiality: (H)igh Risk of leaking critical or sensitive information or data.

Integrity: (N)one

Availability: (N)one

⚙️ We are processing your report and will contact the **scikit-learn** team within 24 hours. 2 years ago

✉️ We have contacted a member of the **scikit-learn** team and are waiting to hear back 2 years ago



Olivier Grisel commented 2 years ago

Maintainer

Thanks for the clear report.

This behavior is intentional as per the docstring:

```
stop_words_ : set
    Terms that were ignored because they either:

    - occurred in too many documents (`max_df`)
    - occurred in too few documents (`min_df`)
    - were cut off by feature selection (`max_features`).

    This is only available if no vocabulary was given.
```

and:

Notes

The `stop_words_` attribute can get large and increase the model size when pickling. This attribute is provided only for introspection and can be safely removed using `delattr` or set to `None` before pickling.

but I agree that users may overlook the security implications w.r.t. unknowingly leaking sensitive information in rare tokens that do not appear as feature names in the output of the model.

Maybe we should add a new constructor parameter to control whether to store the discarded and only do so when the user sets this option explicitly?

Alternatively we could just stop constructing this set entirely (and never store it as an attribute). It can be reconstructed quite easily by users by re-tokenizing their datasets by themselves and doing a set difference with the vocabulary attribute. It's a bit expensive but since this is only necessary for model inspection (typically debugging the choice of the hyper-parameters), I don't expect that many users actually use this attribute.

I don't find our current `stop_words_` attribute very useful anyway because it mixes rare and frequent tokens together and does not store their document frequency values any way.



A **scikit-learn/scikit-learn** maintainer has acknowledged this report 2 years ago



The scheduled publication date was automatically extended from 26th May 2024 to 2nd Jun 2024 due to the maintainers acknowledgement of the report 2 years ago



Olivier Grisel gave praise 2 years ago



The researcher's credibility has slightly increased as a result of the maintainer's thanks: +1



Olivier Grisel commented 2 years ago

Maintainer

I am no CVE taxonomy expert but the Attach Vector set to network seems to be misleading because `TfidfVectorizer` is not bound to a network stack, users might use it completely offline.

Kemal Tugrul commented 2 years ago

Researcher

Hi Oliver,

Thank you for your positive approach and very prompt answer!

This behavior is intentional as per the docstring:

I have read the docs before using the class. But yes, now that I read in retrospective, I can see what it means. I think it could be phrased a bit clearer in the docs what it means.

Also the naming is a bit confusing. Stop words usually refers to very frequently occurring words in a language that does not add information to the sentence. That is why practitioners choose to remove them. I see that there is also a set of that in [sklearn for English](#). However this attribute, `stop_words_` do not fit to the description in my opinion. It contains infrequent (determined by parameters) words/tokens in the dataset. Therefore the term `stop_words` to describe could be confusing.

I can always help with these with a PR.

but I agree that users may overlook the security implications w.r.t. unknowingly leaking sensitive information in rare tokens that do not appear as feature names in the output of the model.

I am glad that we are on the same page. I was very surprised and alerted when I saw it.

Maybe we should add a new constructor parameter to control whether to store the discarded and only do so when the user sets this option explicitly?

Alternatively we could just stop constructing this set entirely (and never store it as an attribute). It can be reconstructed quite easily by users by re-tokenizing their datasets by themselves and doing a set difference with the vocabulary attribute. It's a bit expensive but since this is only necessary for model inspection (typically debugging the choice of the hyper-parameters), I don't expect that many users actually use this attribute.

I think both are good options. In my humble opinion, removing it all together (your second proposal) would be the best. I also do not see any usage of that attribute within the `TfidfVectorizer` class.

I am no CVE taxonomy expert but the Attach Vector set to network seems to be misleading because `TfidfVectorizer` is not bound to a network stack, users might use it completely offline.

I am not an expert either. The thinking is this class would be serialised and stored somewhere in a persistent storage so that a production ML system can read from there upon initialization. But of course this class could exist in development environments or local.

Please let me know what you think.



Olivier Grisel commented 2 years ago

Maintainer

Thanks for the feedback, I started to draft a PR here:

<https://github.com/scikit-learn/scikit-learn/pull/28823>



Olivier Grisel commented 2 years ago

Maintainer

About the CVSS, I think the "Local" attack vector makes more sense. You need access to the model pickle file to read the information. Most of the times, ML models trained on sensitive datasets are not shared in the wild.

The `vocabulary` attribute maybe also hold sensitive data but this is by design, so model developers should be naturally aware of this.

Kemal Tugrul commented 2 years ago

Researcher

Thanks, Oliver.

About the CVSS, I think the "Local" attack vector makes more sense. You need access to the model pickle file to read the information. Most of the times, ML models trained on sensitive datasets are not shared in the wild.

My understanding is that Local means you need to get physical access (e.g. to a laptop) https://en.wikipedia.org/wiki/Common_Vulnerability_Scoring_System. However the models used in production are always stored in a persistent storage in cloud/on-prem production systems/servers.

I am not certain about these so I am happy to change to the right one. Maybe we can get help from Huntr's in-house security researchers for this? What do you think?

The vocabulary attribute maybe also hold sensitive data but this is by design, so model developers should be naturally aware of this.

Yes, I agree. Developers must apply the same privacy requirements they have to this object like they would to any sensitive data.

Kemal Tugrul commented 2 years ago

Researcher

Hi folks - I saw that Oliver merged the PR with the fix. Are we waiting for the release that will have the fix for the next steps? The guidelines mention the "Validation" step:

- **Validate** Once a disclosure is submitted, the maintainer of the vulnerable codebase will be invited to validate or invalidate the vulnerability with the security researcher. If validated, the disclosing researcher will be rewarded a bounty and a CVE will be assigned to the vulnerability (if applicable).

Is that an action point for you Oliver?



Olivier Grisel validated this vulnerability 2 years ago



kemalty has been awarded the disclosure bounty ✓



The fix bounty is now up for grabs



The researcher's credibility has increased: +7



CVE-2024-5206 assigned to this report. 2 years ago



Olivier Grisel marked this as fixed in 1.5.0 with commit 70ca21 2 years ago



Olivier Grisel has been awarded the fix bounty ✓

 We have sent a warning to the **scikit-learn** team to inform them that this report will be published in 48 hours 2 years ago



This vulnerability has now been published 2 years ago



CVE-2024-5206 has now been published 2 years ago



Dan McInerney modified the Severity from Medium (5.3) to Medium (4.7) 2 years ago

[Sign in](#) to join this conversation

CVE

CVE-2024-5206

Published

Vulnerability Type

CWE-921: Storage of Sensitive Data in a Mechanism without Access Control

Severity

Medium (4.7)

Attack vector	Local
Attack complexity	High
Privileges required	Low
User interaction	None
Scope	Unchanged
Confidentiality	High
Integrity	None
Availability	None

[Open in visual CVSS calculator](#) 

Registry

Pypi

Affected Version

1.4.1.post1

Visibility

Public

Status

Fixed

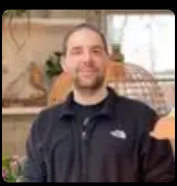
Disclosure Bounty

\$125

Fix Bounty

\$31.25

Found by



Kemal Tugrul

@kemalty

UNPROVEN



Fixed by



Olivier Grisel

@ogrisel

UNPROVEN



Supported by [Palo Alto Networks](#) and Prisma AIRS, leading the way to greater AI security.



© 2024

[Privacy Policy](#)

[Terms of Service](#)

[Code of Conduct](#)

[Cookie Preferences](#)

[Contact Us](#)